Smart Spend Tracker API — документація з обробки помилок

1. Призначення та джерела

Документ описує API Smart Spend Tracker за специфікацією OpenAPI. Специфікація OpenAPI визначає контракт API для Swagger UI; Python-код показує, як оболонка для тестування та розробки в подальшому.

- **OpenAPI: openapi.yaml, версія 3.0.3.**
- **API: FastAPI, версія API 1.0.0.**
- **Формат стандартизованої відповіді про помилку: application/problem+json.**
- **Захищені ендпоїнти використовують Bearer-токен. У поточній демонстраційній реалізації приймається demo-token.**

2. Стандартна структура помилки

Відповідь, сформована обробниками помилок у main.py, має такі поля:

Поле	Тип	Призначення
type	string (URI)	Ідентифікує тип помилки. Формується з коду помилки в нижньому регістрі, символ _ замінюється на -.
title	string	Коротка назва помилки.
status	integer	HTTP-статус.
code	string	Машиночитний код помилки.
detail	string	Короткий опис причини помилки.
instance	string	Шлях запиту, під час якого виникла помилка.
traceId	string	Ідентифікатор для діагностики, наприклад trace-a1b2c3d4e5f6.
errors	object	Необов’язковий словник помилок валідації за полями.
retryAfterSeconds	integer	Необов’язковий час очікування перед повторним запитом.

Приклад відповіді 422:


```
{
 "type": "https://api.smartspendtracker.com/errors/validation-failed",
 "title": "Помилка перевірки даних",
 "status": 422,
 "code": "VALIDATION_FAILED",
 "detail": "Одне або декілька полів не пройшли перевірку.",
 "instance": "/api/expenses",
 "traceId": "trace-a1b2c3d4e5f6",
 "errors": {
  "amount": ["Input should be greater than 0"]
```

```
  }
}
```

traceId генерується випадково для кожної помилки. Повідомлення у errors залежать від того, яке поле не пройшло перевірку.

3. Типи помилок HTTP

3.1. 400 Bad Request — некоректний запит

Значення: запит має некоректний формат або не може бути оброблений як поданий.

У поточному коді: окремого правила для 400 немає. Якщо ендпоїнт вручну підніме HTTPException(400), загальний обробник поверне код HTTP_ERROR, заголовок Помилка запиту та текст detail з винятку. Тому BAD_REQUEST у документації не слід вважати фактично реалізованим кодом відповіді.

3.2. 401 Unauthorized — автентифікація не пройдена

Причини: відсутній токен або переданий неправильний Bearer-токен.

Фактичні поля: code=AUTHENTICATION_REQUIRED, title=Необхідна автентифікація, detail=Потрібно увійти в систему. Додається заголовок WWW-Authenticate: Bearer.

Приклад запиту до захищеного ендпоїнта:

Authorization: Bearer demo-token

Токен є демонстраційним. Реальна перевірка JWT і збереження користувачів не реалізовані; /api/auth/login повертає демонстраційний токен без перевірки email і пароля.

3.3. 403 Forbidden — доступ заборонено

Значення: клієнт автентифікований, але не має дозволу на операцію.

Фактична підтримка: загальний обробник формує RESOURCE_ACCESS_DENIED, Доступ заборонено, Недостатньо прав для операції. Проте в наданому main.py немає системи ролей або перевірок прав, тому наразі не видно сценарію, який самостійно генерує 403.

3.4. 404 Not Found — ресурс не знайдено

Фактичні сценарії: - не знайдено витрату за expenseId; - не знайдено категорію під час оновлення або видалення; - під час створення витрати вказано категорію, якої немає; - під час оновлення витрати вказано неіснуючу категорію.

Обробник повертає code=RESOURCE_NOT_FOUND, title=Ресурс не знайдено, detail=Запитаний ресурс не знайдено. Початкове значення detail з HTTPException замінюється загальним повідомленням.

3.5. 409 Conflict — конфлікт стану ресурсу

Фактичний сценарій: створення категорії з назвою, яка вже існує. Порівняння назв виконується без урахування регістру.

Обробник повертає code=RESOURCE_CONFLICT, title=Конфлікт, detail=Запит конфліктує з поточним станом ресурсу.

Поточний код не перевіряє дублювання назви під час оновлення категорії та не перевіряє використання категорії під час видалення, тому ці сценарії зараз не породжують 409.

3.6. 422 Unprocessable Content — помилка валідації

FastAPI/Pydantic повертає 422, коли параметри або тіло запиту не відповідають моделі.

Основні обмеження з main.py:

Поле	Правило
email	Має бути коректною адресою електронної пошти.
password у реєстрації	Від 8 до 128 символів.
amount	Має бути більше нуля.
currency	Рівно 3 символи.
description	Від 1 до 500 символів.
date	Має бути коректною датою.
categoryId	Має бути UUID або null, якщо поле допускає null.
limit	Ціле число від 1 до 100.
offset	Ціле число, не менше 0.
text для природномовного введення	Від 3 до 2000 символів.
name категорії	Від 1 до 80 символів.

Для помилок автоматичної валідації обробник повертає code=VALIDATION_FAILED, title=Помилка перевірки даних, detail=Одне або декілька полів не пройшли перевірку. та об’єкт errors.

Важлива відмінність: перевірки dateFrom > dateTo у списку витрат і статистиці генерують вручну HTTPException(status_code=422). Через це вони проходять через загальний HTTP-обробник і отримують код HTTP_ERROR, а не VALIDATION_FAILED.

3.7. 429 Too Many Requests — перевищення ліміту

Обробник HTTP-помилок підтримує 429 і повертає RATE_LIMIT_EXCEEDED, Перевищено ліміт запитів, Спробуйте пізніше. Також він читає заголовок Retry-After; якщо його немає або значення не є цілим числом, використовується 60 секунд. Значення додається до retryAfterSeconds.

Обмеження: не налаштовано middleware або інший механізм, який обмежує частоту запитів. Отже, формат відповіді 429 підтримується, але автоматичний rate limiting не реалізований.

3.8. 500 Internal Server Error — внутрішня помилка

OpenAPI визначає 500 як можливу відповідь ендпоїнтів. Однак у наданому main.py немає глобального обробника загальних неочікуваних винятків, який формував би Problem для 500. Тому відповідь 500 не гарантовано матиме описану JSON-структуру.

3.9. 503 Service Unavailable — сервіс недоступний

Фактичний сценарій: POST /api/expenses/natural-language завжди повертає 503, оскільки AI-сервіс не налаштований. У POST /api/expenses/{expenseId}/classify спочатку перевіряється наявність витрати; для наявної витрати ендпоїнт також повертає 503, оскільки AI-сервіс не налаштований.

Обробник повертає SERVICE_UNAVAILABLE, Сервіс недоступний, Необхідний сервіс тимчасово недоступний. Можна передати Retry-After, але поточний код сам не встановлює цей заголовок для 503.

4. Ендпоїнти та очікувані помилки

Таблиця нижче відображає контракт, задекларований у openapi.yaml, після внесених змін. Це список задекларованих відповідей, а не твердження, що всі сценарії вже реалізовані в Python.

Метод	Ендпоїнт	Задекларовані помилки
POST	/api/auth/register	400, 409, 422, 429, 500
POST	/api/auth/login	400, 401, 422, 429, 500
GET	/api/expenses	400, 401, 403, 422, 500
POST	/api/expenses	400, 401, 403, 404, 409, 422, 429, 500
POST	/api/expenses/natural-language	400, 401, 403, 422, 429, 500, 503
GET	/api/expenses/{expenseId}	400, 401, 403, 404, 500
PUT	/api/expenses/{expenseId}	400, 401, 403, 404, 409, 422, 500
DELETE	/api/expenses/{expenseId}	401, 403, 404, 409, 500
GET	/api/categories	401, 403, 500
POST	/api/categories	400, 401, 403, 409, 422, 429, 500
PUT	/api/categories/{category_id}	401, 404, 422, 500
DELETE	/api/categories/{category_id}	401, 404, 500
GET	/api/analytics/summary	400, 401, 403, 422, 429, 500
GET	/api/analytics/by-category	401, 403, 422, 429, 500
POST	/api/expenses/{expenseId}/classify	401, 403, 404, 409, 422, 429, 500, 503
GET	/api/health	Без помилок, описаних у специфікації; 200 при успіху

5. Ендпоїнти категорій, додані до OpenAPI

5.1. PUT /api/categories/{category_id} — оновити категорію

- **Успіх: 200 OK; тіло відповіді містить оновлену категорію.**
- **401: відсутній або недійсний Bearer-токен.**
- **404: категорії з указаним ідентифікатором немає.**
- **422: тіло запиту не відповідає моделі категорії, наприклад name відсутній або порушує обмеження довжини.**

Приклад тіла запиту:

```
{
 "name": "Продукти",
 "color": "#4F46E5"
}
```

У реалізації category_id є рядком, а не UUID. Під час оновлення не перевіряється унікальність назви категорії.

5.2. DELETE /api/categories/{category_id} — видалити категорію

- **Успіх: 200 OK з JSON-повідомленням.**
- **401: відсутній або недійсний Bearer-токен.**
- **404: категорію не знайдено.**

Успішна відповідь:

```
{
 "message": "Category deleted successfully"
}
```

Ендпоїнт повертає 200, а не 204. Код не перевіряє, чи є витрати, пов’язані з категорією.

5.3. GET /api/health — перевірити стан API

Ендпоїнт не потребує автентифікації. У разі успіху повертає 200 OK:

```
{
 "status": "ok"
}
```

Цей ендпоїнт також доданий до OpenAPI, оскільки він є у main.py.

6. Реалізація обробки помилок

HTTPException

handle_http_error() перехоплює HTTPException, визначає код, заголовок і повідомлення для 401, 403, 404, 409, 429 та 503. Для інших статусів використовує HTTP_ERROR, Помилка

запиту і текст початкового detail. Відповідь повертається з типом вмісту application/problem+json. Заголовки винятку зберігаються.

RequestValidationError

handle_validation_error() групує помилки за назвами полів і повертає 422 із кодом VALIDATION_FAILED. Поле errors містить масив повідомлень для кожного поля.

7. Розбіжності та обмеження поточної реалізації

1. **400: немає окремого коду BAD_REQUEST у фактичному обробнику; використовується HTTP_ERROR.**
2. **403: формат відповіді є, але механізм ролей і прав доступу відсутній.**
3. **409: реалізований для повторної назви категорії під час створення; не реалізований для оновлення чи видалення категорії.**
4. **422: автоматична валідація дає VALIDATION_FAILED, ручні HTTPException(422) — HTTP_ERROR.**
5. **429: обробник є, механізму rate limiting немає.**
6. **500: немає загального exception handler, який гарантує формат Problem для неочікуваних помилок.**
7. **503: AI-функціональність наразі недоступна й повертає 503.**
8. **Автентифікація: використовується демонстраційний токен; реальна перевірка облікових даних і JWT не реалізована.**
9. **Зберігання: витрати та категорії зберігаються в оперативній пам'яті, тому не є постійним сховищем.**

8. Як переглянути Swagger UI

1. **Збережіть openapi.yaml у каталозі проєкту.**
2. **Запустіть API з каталогу, де розміщений main.py:**

   python -m uvicorn main:app --reload

3. **Відкрийте http://127.0.0.1:8000/docs.**

FastAPI автоматично генерує Swagger UI зі схем, визначених у Python-коді. Якщо потрібно переглянути саме змінений YAML-файл, відкрийте його в Swagger Editor або налаштуйте FastAPI так, щоб його OpenAPI-схема відповідала цьому файлу.